Московский физико-технический институт
Физтех-школа прикладной математики и информатики

СЛОЖНОСТЬ ВЫЧИСЛЕНИЙ

IV СЕМЕСТР

Лектор: *Мусатов Даниил Владимирович*

$h\nu$

Автор: *Киселев Николай*
Репозиторий на Github

весна 2025

Содержание

1 Класс P **2**
1.1 Базовые определения . . . 2
1.2 Неконструктивные оценки **P** . . . 2
1.3 Другие классы задач . . . 3
1.4 Асимптотики различных задач . . . 3

2 Класс NP **3**
2.1 Определение через сертификат . . . 4
2.2 Некоторые следствия из определений . . . 5
2.3 Задача о рюкзаке и её **NP**-полнота . . . 10

3 Классы задач, связанные с задачами распознавания **11**
3.1 Задача поиска . . . 11
3.2 Задача подсчета . . . 12
3.3 Задача аппроксимации . . . 12

4 Полиномиальная иерархия **12**
4.1 Примеры . . . 13
4.2 Формально . . . 14
4.3 Вопрос о коллапсе иерархии . . . 14
4.4 Полнота в полиномальной иерархии . . . 15
4.5 Альтернирующие Машины Тьюринга . . . 16

1 Класс P

1.1 Базовые определения

Будем рассматривать задачи на распознавание, т.е. дан $A \subset \{0,1\}^*$ и требуется по $x \in \{0,1\}^* \mapsto \begin{cases} 1, x \in A \\ 0, x \notin A \end{cases}$. Пусть M решает данную задачу, т.е. $\forall x (x \in A \Leftrightarrow M(x) = 1)$.

Определение 1.1. $time_M(x)$ — число шагов $M(x)$ при вычислении ответа.

Определение 1.2. $time_M(n) = \max_{x:|x|=n} time_M(x)$

Определение 1.3. $time_M(n) = O(f(n))$, если $\exists C : \forall n : time_M(n) \leqslant C \cdot f(n)$

Возникает вопрос: можно ли сказать, что $time_A(n) = \min_{M:M \text{ решает } A} time_M(n)$? Нет, но показать это достаточно сложно (Теорема Блюма).

Поэтому мы приходим к данному определению:

Определение 1.4. $\mathbf{DTIME}(t(n)) = \{A | \exists M : M(x) = 1 \Leftrightarrow x \in A, time_M(n) = O(t(n))\}$

Заметим, что для определения **DTIME**, необходимо задать модель вычислений. Обычно такой моделью выбирают многоленточную машину Тьюринга.

Определение 1.5. $\mathbf{P} = \mathbf{DTIME}(poly(n)) = \bigcup_{k=1}^{\infty} \mathbf{DTIME}(n^k)$

Тезис Черча-Тьюринга в сильной форме: Любая задача, эффективно решаемая физическим устройстом, решается за полиномиальное время на машине тьюринга.

Пример (Нетривиальные примеры задач из **P**).

1. $\mathbb{P}$ — множество простых чисел
2. Линейное программирование — как пример, нахождения максимума функции на многограннике. Эта задача не бинарная, но вот задача "достижима ли это число на многограннике" принадлежит классу **P**.
3. Симплекс-метод — алгоритм решения

1.2 Неконструктивные оценки P

Рассмотрим, например, задачу определения графа на планарность. Для этого существует два критерия: критерий Понтрягина-Куратовского: граф планарен $\Leftrightarrow$ в нем нет подграфов, гомеоморфных $K_5, K_{3,3}$. Также, существует критерий Вагнера: граф планарен $\Leftrightarrow$ в нем нет миноров $K_5, K_{3,3}$ (минор — граф, полученный из исходного удалением и стягиванием ребер). Рассмотрим свойства, которые сохраняются при удалении и стягивании ребер.

Теорема 1.1 (Робертсона-Сеймура).

1. *Для любого свойства, аналогичного планарности выполнен аналог критерия Вагнера с конечным числом запрещенных миноров.*
2. *Наличие такого минора проверяется за полиномиальное время*

Следствие. Любое такое свойство лежит в классе $\mathbb{P}$.

Но проблема в том, что мы не знаем миноров, которые необходимо проверить, чтобы найти проверить выполнение данного свойства.

1.3 Другие классы задач

Определение 1.6. $\mathbf{QP} = \mathbf{DTIME}(2^{poly(\log(n))}) = \bigcup_{c=1}^{\infty} \mathbf{DTIME}(2^{(\log n)^c})$

Определение 1.7. $\mathbf{E} = \mathbf{DTIME}(2^{O(n)}) = \bigcup_{c=1}^{\infty} \mathbf{DTIME}(2^{cn})$

Определение 1.8. $\mathbf{EXP} = \mathbf{DTIME}(2^{poly(n)}) = \bigcup_{c=1}^{\infty} \mathbf{DTIME}(2^{n^c})$

Определение 1.9. $\mathbf{EE} = \mathbf{DTIME}(2^{2^{cn}}) = \bigcup_{c=1}^{\infty} \mathbf{DTIME}(2^{2^{cn}})$

1.4 Асимптотики различных задач

Пример. $\mathsf{LOG} - \mathsf{CLIQUE} = \{G | \omega(G) \geqslant \log_2 n\} \in \mathbf{QP}$. Является квазиполиномиальной, т.к. $C_n^{\log n} \leqslant n^{\log n}$, т.е. полный перебор осуществляется за квазиполином

Пример (Задача о доминирующем множестве в турнире)**.** непон

Пример. $\mathsf{GI} = \{(G_1, G_2) : G_1 \cong G_2\} \in \mathbf{QP}$ (изоморфность графов).

Пример. $\mathsf{3COL} = \{G : \chi(G) \leqslant 3\} \in E$

Теорема 1.2 (об иерархии по времени)**.** *Если* $f \ll g \Rightarrow \mathbf{DTIME(f(n))} \subsetneq \mathbf{DTIME(g(n))}$

2 Класс NP

Сейчас будем рассматривать модель вычислений — *недетерминированную машину Тьюринга* или НМТ. В отличие от обычной машины Тьюринга, функция перехода теперь многозначна (по аналогии с ДКА и НКА).

Соответственно, время работы такой машины Тьюринга — $time_M(x) = \max \#$шагов по всем вариантам перехода.

Замечание. Можем считать, что дерево переходов двоичное. Действительно, размер ветвления ограничено мощностью $|\Sigma| \cdot |Q| \cdot |\{N, R, L\}|$ — некоторая константа, не зависящая от входа. Тогда каждое m-ветвление можно заменить $\log_2 m$ 2-ветвлениями.

Ответ данной машины будем понимать следующее:

$$M(x) = \begin{cases} 1, \text{существует принимающая ветка} \\ 0, \text{иначе} \end{cases}$$

Замечание. Ответ вычисляется как дизъюнкция по всем результатам работы машины

Определение 2.1. $\mathbf{NTIME}(t(n))$ — класс языков, распознаваемых на НМТ за $O(t(n))$ шагов

Определение 2.2. $\mathbf{NP} = \bigcup_{c=1}^{\infty} \mathbf{NTIME}(n^c)$

Определение 2.3. $\mathbf{NE} = \bigcup_{c=1}^{\infty} \mathbf{NTIME}(2^{cn})$

Определение 2.4. $\mathbf{NEXP} = \bigcup_{c=1}^{\infty} \mathbf{NTIME}(2^{n^c})$

Замечание. $\mathbf{NTIME(t(n))} \subset \mathbf{DTIME(2^{t(n)})}$

Замечание. $\mathbf{NP} \subset \mathbf{EXP}$ — за время $\mathbf{EXP}$ можно построить все дерево и вычислить ответ по определению.

2.1 Определение через сертификат

Теорема 2.1. *$A \in \mathbf{NP} \Leftrightarrow \exists V(x,s)$ — ДМТ, т.ч. $x \in A \Leftrightarrow \exists s : V(x,s) = 1$ и $V(x,s)$ работает за $poly(|x|)$*

Доказательство.

$\Leftarrow$ Рассмотрим следующую НМТ, которая сначала печатает все возможные варианты s (достаточно написать полиномиальное количество символов, т.к. больше V не сможет прочесть), а потом на входе x, s запускает V. Таким образом, Получили машину M, которая в какой-то ветке напечатает нужный сертификат s и выведет $V(x,s) = 1$.

$\Rightarrow$ Возьмем в качестве сертификата код нужной ветви в машине M (0, если надо идти вправо, 1, если влево). Оно и будет нашим сертификатом s. Машина V будет спускаться, в соответствии с сертификатом, по дереву переходов. Тогда сертификат существует $\Leftrightarrow$ сущетсвует принимающая ветвь $\Leftrightarrow A \in \mathbf{NP}$.

$\square$

Упражнение. *Сформулировать и доказать аналогичную теорему для классов* $\mathbf{NE}, \mathbf{NEXP}$

Утверждение 2.1. *1. $A, B \in \mathbf{P} \Rightarrow A \cap B, A \cup B, \overline{A} \in \mathbf{P}$*

2. $A, B \in \mathbf{NP} \Rightarrow A \cap B, A \cup B \in \mathbf{NP}$

Замечание. Вообще говоря, $\overline{A} \in \mathbf{NP}$ — открытый вопрос. Нельзя просто инвертировать значение машины M (пусть мы получим машину $\overline{M}$): тогда $x \in \overline{A} \Leftrightarrow$ все ветки $\overline{M}$ принимающие, а это не то, что мы хотим. Таким образом, мы приходим к следующему определению:

Определение 2.5. $\mathbf{coNP} = \{A \in \{0,1\}^* : \overline{A} \in \mathbf{NP}\}$.

Замечание. Аналогично можно доказать, что $A \in \mathbf{coNP}$ тогда и только тогда, когда ответ вычисляется как конъюнкция всех результатов работы машины или тогда и только тогда, когда $\exists V : x \in A \Leftrightarrow \forall s V(x,s) = 1$ и V вычисляется полиномиально от длины x.

Пример. 1. $\mathsf{SAT} = \{\varphi : \exists x : \varphi(x) = 1\} \in \mathbf{NP}$.

Также имеет смысл рассмотреть двойственную задачу (к задаче опровержимости формулы):

2. $\mathsf{TAUT} = \{\varphi : \exists x : \varphi(x) = 1\} \in \mathbf{coNP}$

Замечание. Несмотря на доказанную теорему о полноте, вывод не будет являться сертификатом. Действительно, вывод, вообще говоря, не обязан быть полиномиальным и, в таком случае, машина не сможет полностью его прочесть (т.к. работает полиномиально от $|x|$)

Отдельный интерес у людей науки представляет множество $(\mathbf{coNP} \cap \mathbf{NP}) \setminus \mathbf{P}$. Рассмотрим следующую задачу:

Определение 2.6. $\mathsf{FACTORING} = \{(n,a,b) : \exists d \in (a,b) : d$ — простое и $n \vdots d\}$

Утверждение 2.2. $\mathsf{FACTORING} \in \mathbf{NP} \cap \mathbf{coNP}$.

Доказательство.

$\in$ **NP** d — сертификат

$\in$ **coNP** сертификат — разложение на простые, каждое из которых $\notin (a, b)$.

□

2.2 Некоторые следствия из определений

Утверждение 2.3. $\mathbf{P} = \mathbf{NP} \Leftrightarrow \mathbf{P} = \mathbf{coNP}$

Утверждение 2.4. *Следует из того, что* $\mathbf{coP} = \mathbf{P}$ *(*$\mathbf{P}$ *замкнут относительно дополнения).*

Замечание. Тем не менее, может быть, что $\mathbf{P} \neq \mathbf{NP}$, но $\mathbf{NP} = \mathbf{coNP}$

Определение 2.7. $A \leqslant_p B$ сводится по Карпу (сводится полиномиально), если $\exists$ всюду полиномиально вычислимая от $|x|$ функция $f(x)$, такая, что $x \in A \Leftrightarrow f(x) \in B$.

Определение 2.8. $\mathsf{INDSET} = \{(G, k) :$ в графе G есть антиклика из k вершин$\}$

Пример. $\mathsf{CLIQUE} \leqslant_p \mathsf{INDSET}$. Действительно, $f(G, k) = f(\overline{G}, k)$ (дополнение по ребрам).

Определение 2.9. $\mathsf{4COL} = \{G : \exists$правильная раскраска в 4 цвета$\}$

Пример. $\mathsf{4COL} \leqslant_p \mathsf{SAT}$. Мы так уже делали на матлоге, когда сводили некоторые задачи к задачам выполнимости формулы. Для каждой вершины заведем две переменные p_i, q_i, отвечающие за цвет. Нам нужно для каждого ребра записать, что две вершины, являющиеся его концами, имеют разный цвет, и взять конъюнкцию, т.е:

$$\bigwedge_{(i,j) \in E} (p_i \neq p_j) \vee (q_i \neq q_j)$$

Размер данной формулы будет полиномиальным относительно размера графа.

Замечание (Свойства $\leqslant_p$)**.**

1. $A \leqslant_p B, B \leqslant_p C \Rightarrow A \leqslant_p C$
2. $A \leqslant_p B, B \in \mathbf{P} \Rightarrow A \in \mathbf{P}$
3. $A \leqslant_p B, B \in \mathbf{NP} \Rightarrow A \in \mathbf{NP}$
4. $A \leqslant_p B \Rightarrow \overline{A} \leqslant_p \overline{B}$

Определение 2.10. Задача $B \in \mathbf{NPH}$ (**NP**-трудной), если $\forall A \in \mathbf{NP} : A \leqslant_p B$.

Определение 2.11. $\mathbf{NPC} = \mathbf{NP} \cap \mathbf{NPH}$ (**NP**-полные)

Следствие.

1. $B \in \mathbf{NPH}, B \leqslant_p C \Rightarrow C \in \mathbf{NPH}$
2. $B \in \mathbf{NPC}, B \leqslant_p C, C \in \mathbf{NP} \Rightarrow C \in \mathbf{NPC}$

Утверждение 2.5.

1. $\mathbf{P} \cap \mathbf{NPH} \neq \varnothing \Rightarrow \mathbf{P} = \mathbf{NP}$
2. $\mathbf{coNP} \cap \mathbf{NPH} \neq \varnothing \Rightarrow \mathbf{NP} = \mathbf{coNP}$

Доказательство.

2. $B \in \mathbf{NPH}, \mathbf{coNP} \Rightarrow \overline{B} \in \mathbf{NP}$. Теперь, если $A \leqslant_p B \Rightarrow \overline{A} \leqslant_p \overline{B}$. Отсюда получаем, что $\overline{A} \in \mathbf{NP}$ и тогда $\mathbf{NP} \subset \mathbf{coNP}$. Тогда:

$$S \in \mathbf{coNP} \Rightarrow \overline{S} \in \mathbf{NP} \Rightarrow \overline{S} \in \mathbf{coNP} \Rightarrow S \in \mathbf{NP}$$

□

Утверждение 2.6. $A \in \mathbf{P}, B, \overline{B} \neq \varnothing \Rightarrow A \leqslant_p B$

Доказательство. Рассмотрим

$$f(x) = \begin{cases} \in B, x \in A \\ \notin B, x \notin A \end{cases}$$

□

Следствие. $\mathbf{P} = \mathbf{NP} \Rightarrow \mathbf{NPC} = P \setminus \{\varnothing, \Sigma^*\}$

Определение 2.12. $\mathsf{TMSAT} = \{(M, x, 1^t) : \exists y \;\; M(x, y)$ и работает за $\leqslant t$ шагов$\}$.

Утверждение 2.7. $\mathsf{TMSAT} \in \mathbf{NP}$

Доказательство. Сертификат — y, верификатор — УМТ □

Утверждение 2.8. $\mathsf{TMSAT} \in \mathbf{NPC}$

Доказательство. Принадлежность **NP** уже доказали, докажем принадлежность **NPC**. Пусть $A \in \mathbf{NP}$. По определению: $x \in A \Leftrightarrow \exists s : V(x, s) = 1$. Положим M — машину Тьюринга, вычисляющую V, $t(n)$ — время работы V для $|x| = n$. Тогда положим $f(x) = (M, x, 1^{t(|x|)})$ и получим, что $A \leqslant_p \mathsf{TMSAT}$. □

Определение 2.13. $\mathsf{3SAT} = \{\varphi | \varphi$ — выполнимая 3-КНФ$\}$.

Теорема 2.2 (Преобразование Цейтина)**.** $\mathsf{SAT} \leqslant_p \mathsf{3SAT}$.

Доказательство. Для каждой подформулы φ_i заведем свою переменную p_i. Тогда достаточно записать, конъюнкцию формул следующего вида: $q_i \equiv q_j * q_k$, если $\varphi_j = \varphi_j * \varphi_k$. Осталось для каждой такой формулы написать 3-КНФ. □

Напоминание. Наша модель вычислений — одноленточную ДМТ, где лента бесконечная вправо. Пусть Γ — ленточный алфавит, Q — множество состояний, $Q \cap \Gamma = \varnothing$. Пусть $|Q \cup \Gamma| \in (2^{k-1}, 2^k]$, тогда будем использовать k бит для кодирования элементов $Q \cup \Gamma$. Конфигурация ДМТ — строка $AqaB, q \in Q, A, B \in \Gamma^*, a \in \Gamma$.

Определение 2.14. Беспрефиксный код — набор слов, где ни одно из слов не является началом другого

Определение 2.15. Беспрефиксное кодирование — функция, такая, что

Пусть также x — вход, $|x| = n$, машина V работает за $\leqslant t(n)$ шагов $\Rightarrow V$ использует $\leqslant t(n)$ ячеек $\Rightarrow$ длина любой конфигурации $\leqslant t(n) + c$. Также будем считать, что машина имеет два завершающих состояния: q_{accept}, q_{reject}, прчием после того, как она пришла в одно из них, она может еще какое-то время передвигаться по ленте, ничего не меняя.

Теорема 2.3 (Кука-Левина)**.** $\mathsf{SAT} \in \mathbf{NPC}$

Доказательство. Рассмотрим V — верификатор SAT. Покажем, как написать следюущую формулу: "из данных x, данных y машина придет в q_a". Рассмотрим следуюшую таблицу (индекс столбца — номер ячейки, индекс строки — индекс в конфигурации):

	0	1	2	...	$t(n)$
0					
1					
2					
$\vdots$					
$t(n)$					

Пусть p_{ij} — k-бит, которые стоят в ij-ой клетке данной таблицы. Запишем формулу, утверждающую, что таблица корректная для V. Тогда наша формула будет иметь вид: $\varphi = \varphi_{start} \wedge \varphi_{accept} \wedge \varphi_{step}$, где:

1. φ_{start} определяет, что изначально машина находилась в корректной конфигурации (изначальная конфигурация $q_1 x\#y\#\ldots\#$), т.е., что $p_0 = q_1 x\#y\#\ldots\#$:

$$\begin{aligned}\varphi_{start} \quad &= \underbrace{(p_{00} = q_1)}_{q_1} \wedge \underbrace{(p_{01} = x_1) \wedge \cdots \wedge (p_{0n} = x_n)}_{x} \wedge \underbrace{(p_{0,n+1} = \#)}_{\#} \wedge \\ &\wedge \underbrace{(p_{0,n+2} = 0 \vee p_{0,n+2} = 1) \wedge \cdots \wedge (p_{0,n+m+1} = 0 \vee p_{0,n+m+1} = 1)}_{y} \wedge \\ &\wedge \underbrace{(p_{0,n+m+2} = \#) \wedge \cdots \wedge (p_{0,t(n)} = \#)}_{\#\cdots\#}\end{aligned}$$

2. φ_{accept} проверяет, что в таблице есть хотя бы одно завершающее состояние, т.е., что наша машина завершилась и приняла слово.

$$\varphi_{accept} = \bigvee_{j=0}^{t(n)} (p_{t(n),j} = q_{accept})$$

3. φ_{step} проверяет, что в машина совершала все переходы корректно. Действительно, для таблицы символ p_{ij} однозначно восстанавливается по $p_{i-1,j-1}, p_{i-1,j}, p_{i-1,j+1}, p_{i-1,j+2}$:

$p_{i-1,j-1}$	$p_{i-1,j}$	$p_{i-1,j+1}$	$p_{i-1,j+2}$
	p_{ij}		

Тогда

$$\varphi_{step} = \bigwedge_{i=1}^{t(n)} \bigwedge_{j=1}^{t(n)} (p_{ij} = f(p_{i-1,j-1}, p_{i-1,j}, p_{i-1,j+1}, p_{i-1,j+2}))$$

Где f — функция, которая восстанавливает по $p_{i-1,j-1}, p_{i-1,j}, p_{i-1,j+1}, p_{i-1,j+2}$ значение p_{ij}.

Тогда формула выполнима $\Leftrightarrow$ V приняло изначальную формулу $\Rightarrow$ получили требуемое □

Определение 2.16. $\mathsf{NAE-3SAT}$ — Дана 3-КНФ. Есть ли набор значений, такой, что в каждой скобке есть истинные и ложные литералы?

Утверждение 2.9. $\mathsf{3SAT} \leqslant_p \mathsf{NAE-3SAT}$

Доказательство. Рассмотрим преобразование:

$$(a_i \vee b_i \vee c_i) \mapsto (a_i \vee b_i \vee x_i) \wedge (\neg x_i \vee c_i \vee z)$$

Где x_i уникально для каждого i, а z одно для всей формулы.

Покажем, что исходная формула φ выполнима тогда и только тогда, когда построенная формула ψ принадлежит $\mathsf{NAE-3SAT}$.

В одну сторону: если φ выполнима, выберем выполняющее назначение и положим $z = 0$. Для каждой скобки $(a_i \vee b_i \vee c_i)$ подберём x_i так, чтобы обе скобки $(a_i \vee b_i \vee x_i)$ и $(\neg x_i \vee c_i \vee 0)$ удовлетворяли условию NAE. Значение x_i можно взять из следующей таблицы истинности:

a_i	$\vee$	b_i	$\vee$	c_i	$(a_i$	$\vee$	b_i	$\vee$	$c_i)$	$\wedge$	$(\neg x_i$	$\vee$	c_i	$\vee$	$z)$
0		0		1	0		0		1		0		1		
0		1		0	0		1		0		1		0		
0		1		1	0		1		1		0		1		
1		0		0	1		0		0		1		0		
1		0		1	1		0		1		0		1		
1		1		0	1		1		0		1		0		
1		1		1	1		1		0		1		1		

Таким образом, полученное назначение даёт решение $\mathsf{NAE-3SAT}$ для ψ.

В обратную сторону: пусть ψ имеет решение в смысле $\mathsf{NAE-3SAT}$. Если в этом решении $z = 1$, то инвертируем все переменные (включая z) — полученный набор также будет решением, поскольку условие NAE инвариантно относительно отрицания. Поэтому можно считать, что $z = 0$. Тогда для каждой исходной скобки $(a_i \vee b_i \vee c_i)$ имеем две скобки: $(a_i \vee b_i \vee x_i)$ и $(\neg x_i \vee c_i \vee 0)$. Из условия NAE для второй скобки при $z = 0$ получаем, что $\neg x_i \vee c_i$ истинно (иначе все литералы были бы ложны). Для первой скобки условие NAE означает, что не все три литерала равны. Совокупность этих условий влечёт истинность $a_i \vee b_i \vee c_i$: действительно, если бы $a_i = b_i = c_i = 0$, то из первой скобки следовало бы $x_i = 1$ (чтобы не все нули), но тогда вторая скобка стала бы $(\neg 1 \vee 0 \vee 0) = (0 \vee 0 \vee 0)$ — все ложны, противоречие. Значит, все исходные скобки истинны, т.е. φ выполнима.

Таким образом, построенное сведение полиномиально и корректно, откуда $\mathsf{NAE-3SAT} \in$ **NP**. □

Утверждение 2.10. $\mathsf{NAE-3SAT} \leqslant_p \mathsf{3COL}$.

Доказательство. По формуле φ с переменными $x_1, \ldots, x_n$ и дизъюнктами $C_1, \ldots, C_m$ (каждый — тройка литералов) строим граф G:

1. Три вершины T,F,B, соединённые попарно (треугольник).
2. Для каждой переменной x вершины x и $\neg x$, соединённые ребром, и каждая из них соединена с B.
3. Для каждого дизъюнкта $C = (l_1 \vee l_2 \vee l_3)$ вершины c_1,c_2,c_3, образующие треугольник, и для каждого i вершина c_i соединена с l_j, l_k, где $\{i,j,k\} = \{1,2,3\}$.

Докажем, что $\varphi \in \mathsf{NAE-3SAT} \iff \chi(G) \leqslant 3$.

$\Rightarrow$ Пусть v — оценка, в каждом дизъюнкте есть и истинный, и ложный литерал. Красим T,F,B в разные цвета. Вершину x красим цветом T, если $v(x) = 1$, и F иначе; $\neg x$ — противоположным цветом. В каждом треугольнике дизъюнкта цвета c_i подбираем так: поскольку среди l_1,l_2,l_3 есть оба цвета, можно назначить c_i цвет, отличный от цветов двух смежных литералов (существование такой раскраски проверяется перебором случаев).

$\Leftarrow$ Пусть G правильно раскрашен в 3 цвета. Можно считать, что цвета T,F,B различны. Тогда x и $\neg x$ имеют цвета T и F (так как оба смежны с B и между собой). Положим $v(x) = 1$, если x цвета T, иначе 0. В каждом дизъюнкте C вершины c_1,c_2,c_3 имеют три разных цвета. Каждая c_i смежна с двумя литералами, поэтому эти литералы не могут иметь цвет c_i. Значит, среди трёх литералов не все одинакового цвета.

Таким образом, сведение корректно и выполняется за полиномиальное время. □

Определение 2.17. EXACTSETCOVER — следующая задача: пусть даны $S_1, \ldots S_m \subset \{1, 2, \ldots n\}$, надо понять, существует ли $i_1, \ldots i_k$ и $\bigsqcup_{j=1}^{k} S_{i_k} = \{1, \ldots m\}$

Утверждение 2.11. $\mathsf{3SAT} \leqslant_p \mathsf{EXACTSETCOVER}$.

Доказательство. По формуле φ с переменными $x_1, \ldots, x_n$ и дизъюнктами $C_1, \ldots, C_m$ (каждый — тройка литералов) строим множество U и семейство подмножеств $\mathcal{F}$.

1. Элементы: для каждой переменной x_i: x_i и $\neg x_i$; для каждого дизъюнкта C_j: c_{j1},c_{j2},c_{j3} и d_{C_j}.
2. Множества:
   (a) Для каждого литерала l (т.е. x_i или $\neg x_i$) множество $L_l = \{l\} \cup \{c_{jk} \mid$ литерал l входит в C_j на
   (b) Для каждого дизъюнкта C_j и каждого непустого $T \subseteq \{1,2,3\}$, $T \neq \{1,2,3\}$ (т.е. $|T| \leqslant 2$), множество $M_{j,T} = \{d_{C_j}\} \cup \{c_{jk} \mid k \in T\}$.

Утверждаем: φ выполнима $\iff$ существует точное покрытие U выбранными множествами.

$\Rightarrow$ Пусть v — выполняющая оценка. Выберем L_{x_i}, если $v(x_i) = 1$, иначе $L_{\neg x_i}$. Эти множества покрывают все элементы x_i,$\neg x_i$ и все c_{jk}, соответствующие истинным литералам. Для каждого C_j пусть S_j — множество позиций истинных литералов. Тогда $S_j \neq \varnothing$. Пусть $T_j = \{1,2,3\} \setminus S_j$ — позиции ложных литералов ($|T_j| \leqslant 2$). Выберем M_{j,T_j} (если $T_j = \varnothing$, берём $M_{j,\varnothing}$ — множество $\{d_{C_j}\}$). Эти множества попарно не пересекаются с выбранными L и между собой и покрывают все оставшиеся c_{jk} и d_{C_j}.

$\Leftarrow$ Пусть есть точное покрытие $\mathcal{C}$. Для каждого i ровно одно из L_{x_i},$L_{\neg x_i}$ принадлежит $\mathcal{C}$ (иначе x_i или $\neg x_i$ не покрыты). Задаём $v(x_i) = 1$, если $L_{x_i} \in \mathcal{C}$, иначе 0. Для каждого C_j элемент d_{C_j} покрыт некоторым $M_{j,T}$, $T \subseteq \{1,2,3\}$, $|T| \leqslant 2$. Тогда c_{jk} при $k \in T$ покрыты этим $M_{j,T}$, а c_{jk} при $k \notin T$ должны быть покрыты L_l, т.е. соответствующие литералы истинны. Так как $T \neq \{1,2,3\}$, имеем $k \notin T$ хотя бы для одного k, значит, в C_j есть истинный литерал.

Следовательно, сведение корректно и полиномиально. □

2.3 Задача о рюкзаке и её NP-полнота

Определение 2.18.

$$\mathsf{KNAPSACK} = \left\{ (w_1, \dots w_n, W, p_1, \dots p_n, P) : \exists \alpha \in \{0,1\}^n : \sum_{i=1}^{n} \alpha_i w_i \leqslant W, \sum_{i=1}^{n} \alpha_i p_i \geqslant Ps \right\}$$

По сути, по набору предметов w_i, p_i, где первое — вес, второе — стоимость требуется понять, сможем ли мы их поместить в рюкзак вместимости W, чтобы суммарная стоимость была не меньше, чем P.

Определение 2.19. $\mathsf{SUBSET-SUM} = \{(m_1, \dots m_n, M) : \exists \alpha \in \{0,1\}^n : \sum_{i=1}^{n} \alpha_i m_i = M\}$

Замечание. При $p_i = w_i = m_i, P = W = M$ задача $\mathsf{SUBSET-SUM}$ эквивалентна задаче $\mathsf{KNAPSACK}$.

Определение 2.20. Частный случай задачи B — это $B \cap S$ для некоторого $S \in P$.

Утверждение 2.12. *Частный случай* $B \cap S \in \mathbf{NPC} \Rightarrow B \in \mathbf{NPC}$.

Доказательство. Зафиксируем $A \in \mathbf{NP}$. Из определения **NP**-полноты, существет такая f, что:

$$x \in A \Leftrightarrow f(x) \in B \cap S$$

Рассмотрим (пусть $x_0 \notin B$):

$$f'(x) = \begin{cases} f(x), f(x) \in S \\ x_0, f(x) \notin S \end{cases}$$

Эта функция доказывает $A \leqslant_p B$ □

Утверждение 2.13. $\mathsf{EXACTSETCOVER} \leqslant_p \mathsf{SUBSET-SUM}$.

Доказательство. Рассмотрим для каждого S_i строку $a_i = \underbrace{0100\dots010}_{n}$, где $a_{ij} = 1 \Leftrightarrow j \in S_i$ (по сути, маску множества S_i). Рассмотрим данную строку как число в $(n+1)$-ичной системе счисления. Положим $M = \underbrace{11\dots1}_{n}$ и рассмотрим его как число в $(n+1)$-ичной системе счисления. Тогда существует дизъюнктное покрытие множествами $\Leftrightarrow \exists a_{i_1} + \dots + a_{i_k} = M$ (перехода через разряд не будет, поэтому в каждом разряде должно быть ровно по одному a_i, который его заполняет). □

Определение 2.21. $\mathsf{SETCOVER}$ — то же самое, что и $\mathsf{EXACTSETCOVER}$, только S_i могут пересекаться, но на их количество есть ограничение.

Определение 2.22. $\mathsf{HITTING-SET} = \{(S_1, \dots S_n, k) : \exists X = \{x_1, \dots x_k\} : \forall i : X \cap S_i \neq \varnothing\}$

Утверждение 2.14. $\mathsf{HITTING-SET} \in \mathbf{NPC}$.

Первое доказательство. Является частным случаем задачи VERTEXCOVER □

Второе доказательство. $\mathsf{3SAT} \leqslant_p \mathsf{HITTING-SET}$ □

Доказательство. Переменной p сопоставим множество $\{p_0, p_1, p\}$. Скобке $(l_1 \vee l_2 \vee l_3)$ сопоставим множество $\{l_1, l_2, l_3\}$. Также положим k — количество переменных. Нетрудно проверить, что данное преобразование реализует полиномиальное сведение. □

Определение 2.23. QUADEQ — системы квадратных уравнений. Пусть $A_1, \dots A_k \in \{0,1\}^{n \times n}$, $b_1, \dots b_k \in \{0,1\}$. Задача состоит в том, существует ли $(x_1, \dots x_n)^T$ такой, что $\forall i = 1, \dots k$: $x^T A_i x = b_i$

Утверждение 2.15. $\mathsf{SAT} \leqslant_p \mathsf{QUADEQ}$

Доказательство. Доказательство аналогично преобразованию Цейтина. Для каждой подформулы заводим новую переменную и для нее пишем многочлен жегалкина второй степени:

$$\begin{array}{cc} p = q \wedge r & p^2 \oplus qr = 0 \\ p = \neg q & p^2 \oplus q^2 = 1 \\ p = q \vee r & p^2 \oplus q^2 \oplus r^2 \oplus qr = 1 \end{array}$$

И для переменной, отвечающей за исходную формулу пишем уравнение $w^2 = 1$. □

3 Классы задач, связанные с задачами распознавания

3.1 Задача поиска

Рассмотрим задачу поиска: по полиномиальному предикату $V(x, y)$ и данному x требуется найти y такой, что $V(x, y) = 1$ или, в случае, если таких нет, запустить игру Марио.

Однако, в общем случае не всегда задача поиска и распознавания не всегда одинаковы по сложности

Пример. Рассмотрим следующую задачу поиска: "по данному n найти его какой-нибудь простой делитель" и задачу распознавания: "по данному n определить, существуют ли у него простые делители". У второй задачи ответ очевиден ($n = 1$ — нет, иначе — да), первая задача эквивалентна FACTORING, сложность которой не известна.

Теорема 3.1. *Любая* **NPC** *задачи поиска сводится к задаче распознавания в смысле сводимости по Куку: используем решатель задачи распознавания как к оракулу, запускаем его много раз и проводим некоторые вычисления с ответами.*

Пример. Рассмотрим задачу о клике: $x = (G, k), y = S \subset V(G)$ и $V(x, y) = \begin{cases} 1, |y| = k, y \text{ клика} \\ 0, \text{ иначе} \end{cases}$.

Рассмотрим следующую процедуру:

```
find_clique(G, k) {
    if (G,k) ∉ CLIQUE {
        return ⊥
    }

    v = random vertex from G

    if (G \ {v}, k) ∈ CLIQUE {
        return find_clique(G \ {v}, k)
    } else {
        return find_clique(G \ {v}, k − 1) ∪{v}
    }
}
```

3.2 Задача подсчета

Также имеет смылс рассмотреть задачи подсчета: по полиномиальному предикату $V(x, y)$ и данному x требуется найти количество y таких, что $V(x, y) = 1$.

Пример. Пусть A — матрица размера $n \times n$ из нулей и единиц.

$$Perm\ \ A = \sum_{\sigma \in S_n} \prod_{i=1}^{n} A_{i\sigma(i)}$$

По сути, это задача о нахождении количества совершенных парсочетаний для графа, для которого A задает матрицу Татта (матрица смежности между двумя долями). Вопрос о нахождении паросочетания можно находить полиномиально, но вот данная задача уже сложная.

Теорема 3.2. *Подсчет $Perm\ \ A$ — **NPC**-задача*

3.3 Задача аппроксимации

Пусть дано x. Требуется найти максимальное (минимальное) значение $f(x, y)$ по y. Задача оптимизации для данной задачи звучит так: найти y такой, что $f(x, y) \begin{matrix} \leqslant \rho \cdot \max_y f(x, y) \\ \geqslant \rho \cdot \min_y f(x, y) \end{matrix}$

И может так случиться, что задача из **NPC**, но ее приближение полиномиально.

Пример. **NPC**-задача: найти вершинную покраску. Приближение: ищем с фактором 2. Достаточно выбрать максимальное паросочетание, берем все вершины и ребра из него.

Однако, бывает так, что даже приближение задачи **NP**-трудно.

4 Полиномиальная иерархия

Определение 4.1. $\mathbf{DP} = \{A \cap B : A \in \mathbf{NP}, B \in \mathbf{coNP}\} = \{A \setminus B : A \in \mathbf{NP}, B \in \mathbf{NP}\}$

4.1 Примеры

Вспомним арифметическую иерархию. Мы строили ПНФ для формул, которые являются вычислимыми. Построим полиномиальную иерархию: оставшаяся часть после кванторов должна быть полиномиально вычислима Рассмотрим следующие задачи:

1. **Задача о минимизации формулы.** $\mathsf{MINEQFORM} = \{(\varphi, k) : \exists\psi \sim \varphi, |\psi| \leqslant k\}$. В кванторах эту задачу можно записать так: $\exists\psi\forall x(|\psi| \leqslant k \wedge \psi(x) = \varphi(x))$. Заметим, что эта формула в ПНФ, причем начинается с $\exists$.

2. **Задачи точной оптимизации.** Например: $\mathsf{CHROMNUM} = \{(G, k) : \chi(G) = k\}$. В кванторах эту задачу можно записать так: $\exists$ правильная раскраска в k цветов $\wedge$ $\forall$ раскраска в $k-1$ цвет неправильная. Её можно привести к ПНФ, причем начинаться она будет с двух кванторов. Порядок кванторов в данном случае не имеет значения

3. **Обобщенная Рамсеевость.** Число Рамсея определяется следующим образом: $R(s,t) = \min\{N :$ в любом графе на N вершинах есть s-клика или t-независимое множество$\}$. Обобщенная Рамсеевость, же задает следующий вопрос: пусть даны 3 графа G, H_1, H_2, верно ли что при любой раскраске G в 2 цвета найдется подграф первого цвета, изоморфный H_1 или подграф второго цвета, изоморфный H_2? Данная задача приводится к ПНФ следующим образом:

$$\forall \text{ раскраски } \exists \text{ подграф, изоморфизм } \ldots$$

   Здесь уже формула начинается с $\forall$.

4. **Кликовая раскраска.** Существует ли кликовая раскраска графа в k цветов, т.е. такая, что в любой максимальной клике есть вершины разного цвета. Таким образом, в ПНФ будет 2 квантора:

$$\exists \text{ раскраска } \forall \text{ максимальной клики } \underbrace{\exists \text{ 2 вершины}}_{\text{полиномиально вычислимо}} \ldots$$

5. **Наследуемая кликовая раскраска.** Аналогично предыдущему, но утверждение должно выполняться для любого подграфа. ПНФ:

$$\forall \text{ подграфа } \exists \text{ раскраска } \forall \text{ максимальной клики } \ldots$$

Определение 4.2. Пусть $U = \{1, \ldots n\}$ и дана система множеств $\{S_1, \ldots S_m\}, S_i \subset U$. Тогда: $\dim_{VC}(S_1, \ldots S_m) = \max\{|T| : \forall X \subset T : \exists i : S_i \cap T = X\}$. Таким образом: $\mathsf{VCDIM} = \{(S_1, \ldots S_m, d) : \dim_{VC}(S_1, \ldots S_m) \geqslant d\}$.

5. **Размерность Вапника-Червоненкиса.** Задача VCDIM, но с неявным заданием формулы (чтобы можно было записать экспоненциальное количество множеств) через формулу $\varphi(i, x) = \begin{cases} 1, x \in S_i \\ 0, x \notin S_i \end{cases}$.

4.2 Формально

Определение 4.3. $\mathbf{\Sigma_k^p}$ — класс языков A, таких, что $\exists V$ — полиномиально вычислимая от длины x, такая, что:

$$x \in A \Leftrightarrow \exists y_1 \forall y_2 \ldots \overset{\forall}{\exists} y_k V(x, y_1, \ldots y_k)$$

Определение 4.4. $\mathbf{\Pi_k^p}$ — класс языков A, таких, что $\exists V$ — полиномиально вычислимая от длины x, такая, что:

$$x \in A \Leftrightarrow \forall y_1 \exists y_2 \ldots \overset{\forall}{\exists} y_k V(x, y_1, \ldots y_k)$$

Замечание.

1. $P = \Sigma_0^p = \Pi_0^p$
2. $\Sigma_i^p, \Pi_i^p \subset \Sigma_{i+1}^p, \Pi_{i+1}^p$
3. $\Sigma_1^p = \mathbf{NP}, \Pi_1^p = \mathbf{coNP}$
4. $\Sigma_1^p, \Pi_1^p \subset \mathbf{DP} \subset \Sigma_2^p, \Pi_2^p$

Определение 4.5.

$$\mathbf{PH} = \bigcup_{k=0}^{\infty} \mathbf{\Sigma_k^p} = \bigcup_{k=0}^{\infty} \mathbf{\Pi_k^p}$$

4.3 Вопрос о коллапсе иерархии

Возникает вопрос: а почему $\mathbf{PH} \neq \mathbf{P}$?

Теорема 4.1. $\mathbf{P} = \mathbf{NP} \Rightarrow \mathbf{P} = \mathbf{PH}$

Идея доказательства. $\mathbf{P} = \mathbf{NP} \Rightarrow \mathbf{P} = \mathbf{coNP}$. Но тогда мы можем убирать последние кванторы в формуле и, таким образом, $\Sigma_i^p, \Pi_i^p = \mathbf{P}$. □

Замечание. $A, B \in \mathbf{\Sigma_k^p} \Rightarrow A \cup B, A \cap B \in \mathbf{\Sigma_k^p}, A \in \mathbf{\Sigma_k^p} \Leftrightarrow \overline{A} \in \mathbf{\Pi_k^p}$

Доказательство. $A \in \mathbf{\Sigma_k^p} \Leftrightarrow \overline{A} \in \mathbf{\Pi_k^p}$ — очевидно из отрицания к формуле A. Докажем, что $A \cup B \in \mathbf{\Sigma_k^p}$. Это следует из следующего (если φ, ψ не зависят от y, x соответственно):

$$\forall x \varphi(x) \vee \forall y \psi(y) \leftrightarrow \forall (x, y) \varphi \vee \psi$$

Утверждение $A \cap B \in \mathbf{\Sigma_k^p}$ получается и равенства $A \cap B = \overline{\overline{A} \cup \overline{B}}$. □

Теорема 4.2 (Об условиях коллапса)**.** *Следующие условия эквивалентны:*

1. $\mathbf{\Sigma_k^p} = \Sigma_{k+1}^p$

1'. $\mathbf{\Pi_k^p} = \Pi_{k+1}^p$

2. $\mathbf{\Sigma_k^p} = \mathbf{\Pi_k^p}, k > 0$

3. $\mathbf{PH} = \mathbf{\Sigma_k^p}$

Доказательство.

(3) $\Rightarrow$ (1) $\mathbf{\Sigma_k^p} \subset \Sigma_{k+1}^p \subset \mathbf{PH} = \mathbf{\Sigma_k^p}$

(1) $\Rightarrow$ (2) $\mathbf{\Pi_k^p} \subset \Sigma_{k+1}^p = \mathbf{\Sigma_k^p} \Rightarrow \mathbf{\Pi_k^p} = \mathbf{\Sigma_k^p}$. Теперь:

$$A \in \mathbf{\Sigma_k^p} \Rightarrow \overline{A} \in \mathbf{\Pi_k^p} \Rightarrow \overline{A} \in \mathbf{\Sigma_k^p} \Rightarrow A \in \mathbf{\Pi_k^p} \Rightarrow \mathbf{\Sigma_k^p} = \mathbf{\Pi_k^p}$$

(2) $\Rightarrow$ (3) Рассмотрим $A \in \Sigma_{k+1}^p$

$$x \in A \Leftrightarrow \exists y_1 \forall y_2 \ldots \overset{\forall}{\exists} y_{k+1} V(x, y_1, \ldots y_{k+1})$$

Рассмотрим теперь язык

$$Z = \{(x, y_1) : \forall y_2 \ldots \overset{\forall}{\exists} y_{k+1} V(x, y_1, \ldots y_{k+1})\} \in \mathbf{\Pi_k^p} \Rightarrow \in \mathbf{\Sigma_k^p}$$

Но тогда:

$$Z = \{(x, y_1) : \exists z_1 \ldots \overset{\forall}{\exists} z_k W(x, z_1, \ldots z_k)\}$$

Из чего заключаем:

$$x \in A \Leftrightarrow \underbrace{\exists y_1 \exists z_1}_{\exists (y_1, z_1)} \ldots \overset{\forall}{\exists} z_k W(x, z_1, \ldots z_k)$$

Получили, что $A \in \mathbf{\Sigma_k^p}$. Тогда $\Sigma_{k+1}^p = \Pi_{k+1}^p = \mathbf{\Sigma_k^p}$. Отсюда по индукции получаем, что $\Sigma_{k+l} = \Pi_{k+l} = \mathbf{\Sigma_k^p} \Rightarrow \mathbf{PH} = \Sigma_k$

□

Замечание. $A \leqslant_p B, B \in \mathbf{\Sigma_k^p} \Rightarrow A \in \mathbf{\Sigma_k^p}$.

4.4 Полнота в полиномальной иерархии

Можно по аналогии определить $\mathbf{\Sigma_k^p}$-полные, $\mathbf{\Pi_k^p}$-полные и $\mathbf{PH}$-полные задачи:

Определение 4.6. $A \in \mathbf{\Sigma_k^p}$ называется полной, если $\forall B \in \mathbf{\Sigma_k^p} : B \leqslant_P A$

Определение 4.7. $A \in \mathbf{\Pi_k^p}$ называется полной, если $\forall B \in \mathbf{\Pi_k^p} : B \leqslant_P A$

Определение 4.8. $A \in \mathbf{PH}$ называется полной, если $\forall B \in \mathbf{PH} : B \leqslant_P A$

Замечание. Из теоремы об условиях коллапса, получаем, что $\mathbf{PH}$-полные задачи существуют в том и только том случае, когда иерархия коллапсирует.

Обобщим следующие задачи для классов $\mathbf{P}, \mathbf{NP}$:

$$\mathsf{SAT} = \{\varphi | \exists x : \varphi(x) = 1\}$$

$$\mathsf{TAUT} = \{\varphi | \forall x : \varphi(x) = 1\}$$

Таким образом, просто добавим кванторы:

$$\Sigma_k\mathsf{SAT} = \left\{\varphi | \exists x_1 \forall x_2 \ldots \overset{\forall}{\exists} x_k : \varphi(x)\right\}$$

$$\Pi_k\mathsf{SAT} = \left\{\varphi | \exists x_1 \forall x_2 \ldots \overset{\forall}{\exists} x_k : \varphi(x)\right\}$$

Можно смотреть на данную задачу, как на "Игру в формулу": Алиса и Боб играют по следующим правилам: Алиса выбирает x_1, Боб выбирает x_2 и так далее. Если в итоге получилось, что $\varphi = 1$, то победила Алиса, иначе — Боб (для задачи $\Pi_k\mathsf{SAT}$ – наоборот). Таким образом, задача $\Sigma_k\mathsf{SAT}$, по сути, утверждает, что у Алисы есть выигрышная стратегия, а $\Pi_k\mathsf{SAT}$ — что у Боба.

Лемма 4.1. $\Sigma_k\mathsf{SAT}$ *полна в* $\mathbf{\Sigma_k^p} \Leftrightarrow \Pi_k\mathsf{SAT}$ *полна в* $\mathbf{\Pi_k^p}$

Доказательство. Достаточно доказать только в одну сторону (в другую аналогично). Докажем вправо. $\varphi \notin \Sigma_k\mathsf{SAT} \Leftrightarrow \neg\varphi \in \Pi_k\mathsf{SAT}$. Тогда, если $A \in \mathbf{\Pi_k^p} \Rightarrow \overline{A} \in \mathbf{\Sigma_k^p}$. Из этого следует, что $\overline{A} \leqslant_p \Sigma_k\mathsf{SAT}$. Но тогда:

$$x \in \overline{A} \Leftrightarrow f(x) \in \Sigma_k\mathsf{SAT}$$

$$x \in A \Leftrightarrow f(x) \notin \Sigma_k\mathsf{SAT}$$

$$x \in A \Leftrightarrow f(x) \in \Pi_k\mathsf{SAT}$$

$$A \leqslant_P \Pi_k\mathsf{SAT}$$

□

Лемма 4.2. $\Sigma_k\mathsf{SAT}$ *полно в* $\mathbf{\Sigma_k^p}$ *при нечетных* k. $\Pi_k\mathsf{SAT}$ *полно в* $\mathbf{\Sigma_k^p}$ *при четных* k.

Доказательство. Пусть k нечетно, $A \in \mathbf{\Sigma_k^p}$. $x \in A \Leftrightarrow \exists y_1 \forall y_2 \ldots \exists y_k V(x, y_1, \ldots y_k)$. В таком случае сводимость будет точно такая же как и в теореме Кука-Левина, только последние два получившиеся квантора $\exists x_k \exists$(заполнение таблицы) мы склеим в один. □

Замечание. Для четных k уже не получится так же доказать, что $\Sigma_k\mathsf{SAT}$ полно в $\mathbf{\Sigma_k^p}$ (т.к. последние кванторы не склеятся в один) и придется использовать первую лемму

Таким образом, мы получаем следствие:

Следствие. $\Sigma_k\mathsf{SAT}, \Pi_k\mathsf{SAT}$ полны в $\mathbf{\Sigma_k^p}, \mathbf{\Pi_k^p}$ соответственно.

Следствие. $\forall k \exists \mathbf{\Sigma_k^p}$-полные и $\mathbf{\Pi_k^p}$-полные задачи.

4.5 Альтернирующие Машины Тьюринга

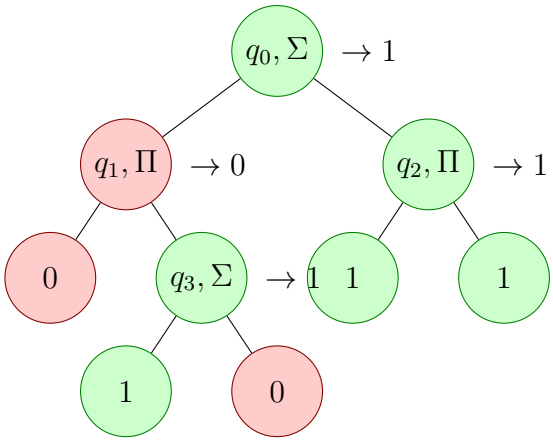
Определение 4.9. Альтернирующая машина Тьюринга — это такая, у которой

1. Функция переходов многозначная
2. Состояния подразделяются на Σ-состояния и Π-состояния

Вычисление ответа машины происходит следующим образом: заметим, что по завершении работы, получается некоторое дерево вычисления нашей машины. Результат вычислений определяем следующим образом:

1. Для листа q_{accept} означат результат 1, а q_{reject} — результат 0
2. Для произвольной вершины в случае Σ-состояния результатом будет дизъюнкция ответов всех её сыновей, иначе — конъюнкция.

Пример (Вычисление ответа на Альтернирующей Машине Тьюринга)**.**



Замечание. НМТ — частный случай АМТ (достаточно сделать все состояния Σ-состояниями).

Время работы АМТ, соответственно, определяется как длина самой большой ветви. Число альтернаций — максимальное количество переключений $\Sigma \leftrightarrow \Pi$.

Определим следующие классы языков:

Определение 4.10. $\mathbf{\Sigma_k TIME}(t(n))$ — класс языков, распознаваемых наа АМТ с начальным состоянием Σ, временем работы $O(t(n))$ и числом альтернаций $\leqslant k$

Определение 4.11. $\mathbf{\Pi_k TIME}(t(n))$ — класс языков, распознаваемых наа АМТ с начальным состоянием Π, временем работы $O(t(n))$ и числом альтернаций $\leqslant k$

Утверждение 4.1.

$$\mathbf{\Sigma_k^p} = \bigcup_{c=1}^{\infty} \mathbf{\Sigma_k TIME}(n^c)$$

$$\mathbf{\Pi_k^p} = \bigcup_{c=1}^{\infty} \mathbf{\Pi_k TIME}(n^c)$$

Утверждение 4.2. *Докажем для* $\mathbf{\Sigma_k^p}$

$\subset$ *Рассмотрим следующую АМТ: в Σ-состоянии пишем y_1, переходим в Π-состояние, пишем в нем y_2 и так далее.*

$\supset$ *Положим y_1 — номер ветви вдоль Σ-состояния, y_2 — номер ветви вдоль Π-состояни и так далее. Машина V будет просто вычислять результат вдоль ветви $y_1, \ldots y_k$.*

А что если время работы полиномиально, но число альтернаций не ограничено? Такой класс называется **AP** и он совпадает с классом **PSPACE**, т.е. с классом языков, распознаваемых на полиномиальной памяти.